

**PRACTICE EXERCISES OF THE MICROPROCESSORS
& MICROCONTROLLERS**

Instructor: The Tung Than

Student's name: Gia Hung Nguyen

Student code: 24520603

PRACTICE REPORT NO 6
IO PROCESSING, CALCULATION AND
MEMORY ON THE 8086 PROCESSOR

- I. Yêu cầu bài thực hành**
- II. Phiên bản 1: Chương trình sử dụng mảng**
- III. Phiên bản 2: Chương trình sử dụng memory**
- IV. So sánh hai phiên bản chương trình**
- V. Kết quả mô phỏng**

I. Yêu cầu bài thực hành

Bài thực hành yêu cầu viết chương trình Assembly 8086 thực hiện các chức năng sau:

- Nhập một số N có 2 chữ số từ bàn phím.
- Kiểm tra điều kiện: $9 < N < 100$.
- In ra màn hình N số Fibonacci đầu tiên.
- Thực hiện chương trình bằng hai cách khác nhau:
 - Phiên bản 1: sử dụng mảng để lưu trữ và tính toán.
 - Phiên bản 2: sử dụng các vùng nhớ riêng lẻ thay cho mảng.

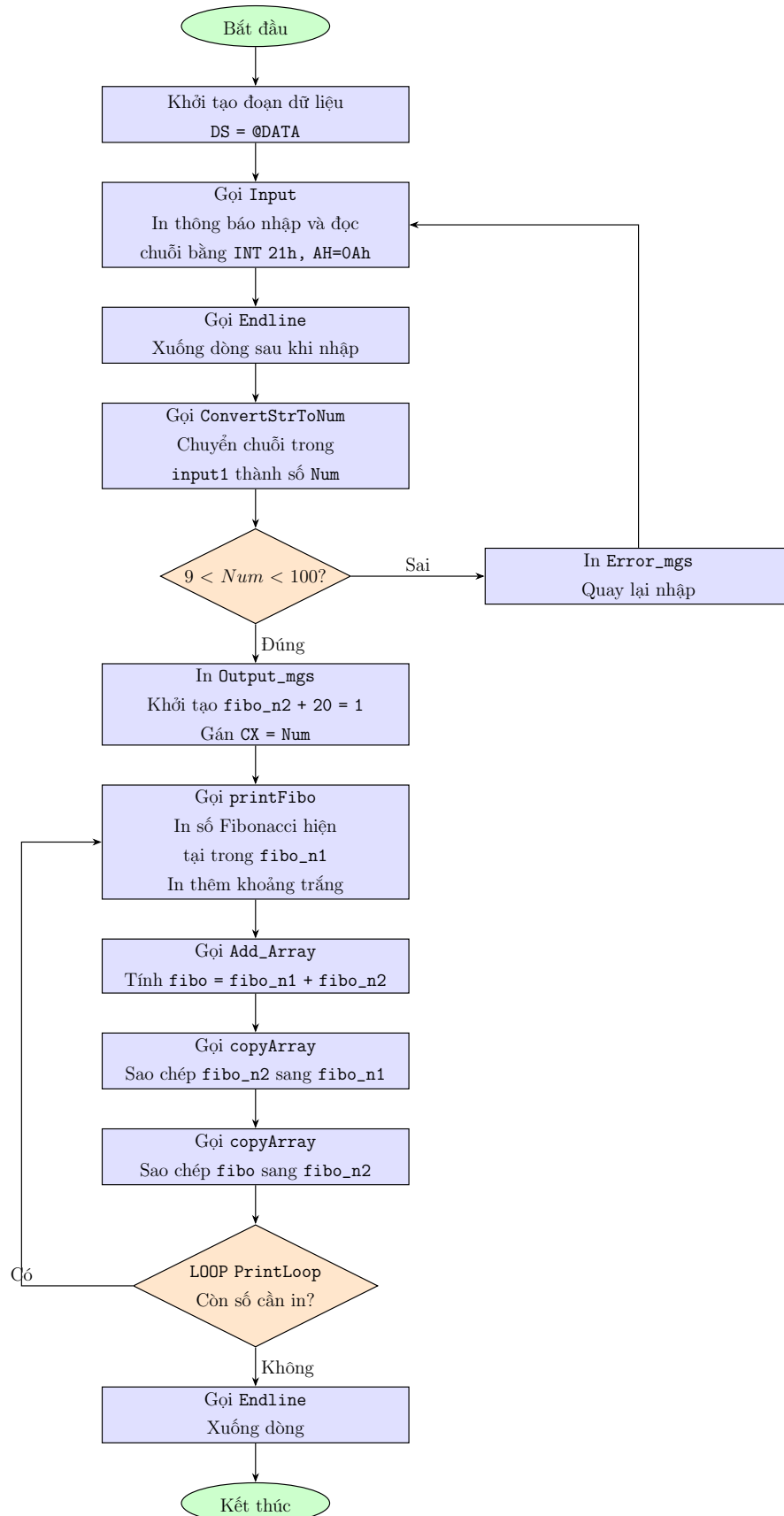
Dãy Fibonacci được xác định theo công thức:

$$F_0 = 0, \quad F_1 = 1, \quad F_i = F_{i-1} + F_{i-2} \quad (i \geq 2)$$

Vì các số Fibonacci tăng rất nhanh và có thể vượt quá giới hạn lưu trữ của thanh ghi 16-bit, chương trình cần sử dụng kỹ thuật lưu trữ số lớn trong bộ nhớ. Báo cáo trình bày hai phiên bản chương trình: phiên bản dùng mảng và phiên bản dùng các biến WORD riêng lẻ.

II. Phiên bản 1: Chương trình sử dụng mảng

1. Flowchart chương trình sử dụng mảng



Hình 1. Flowchart chương trình Fibonacci sử dụng mảng

2. Nguyên lý hoạt động

Ở phiên bản sử dụng mảng, chương trình biểu diễn mỗi số Fibonacci bằng một mảng gồm 21 phần tử kiểu byte. Mỗi phần tử lưu một chữ số thập phân, vì vậy chương trình có thể xử lý được các số Fibonacci lớn hơn giới hạn 16-bit của vi xử lý 8086.

Ba mảng chính được sử dụng là `fibo_n1`, `fibo_n2` và `fibo`. Trong đó, `fibo_n1` lưu số Fibonacci hiện tại, `fibo_n2` lưu số Fibonacci kế tiếp, còn `fibo` lưu kết quả cộng của hai số trước đó.

Ban đầu, các phần tử của ba mảng đều bằng 0. Chương trình gán phần tử cuối cùng của mảng `fibo_n2` bằng 1 thông qua lệnh:

```
MOV fibo_n2 + 20, 1
```

Vì các chữ số được lưu từ trái sang phải, phần tử cuối cùng của mảng là hàng đơn vị. Do đó, sau khi khởi tạo ta có:

$$fibo_n1 = 0, \quad fibo_n2 = 1$$

Quá trình nhập dữ liệu được thực hiện trong thủ tục `Input`. Thủ tục này in chuỗi nhắc nhập `Input_mgs`, sau đó dùng ngắt `INT 21h`, `AH = 0Ah` để đọc chuỗi từ bàn phím vào bộ đệm `input1`. Sau khi nhập xong, chương trình gọi `Endline` để xuống dòng.

Tiếp theo, chương trình gọi `ConvertStrToNum`. Thủ tục này duyệt từng ký tự trong bộ đệm `input1`, nhân kết quả hiện tại với 10 rồi cộng thêm chữ số mới. Kết quả cuối cùng được lưu vào biến `Num`.

Sau khi có `Num`, chương trình kiểm tra điều kiện:

$$9 < Num < 100$$

Nếu `Num` không hợp lệ, chương trình nhảy đến nhãn `InvalidInput`, in thông báo lỗi `Error_mgs` rồi quay lại nhãn `InputAgain` để nhập lại. Nếu hợp lệ, chương trình in thông báo kết quả `Output_mgs`, khởi tạo `fibo_n2 + 20 = 1`, sau đó đưa số lượng phần tử cần in vào thanh ghi `CX` bằng lệnh:

```
MOV CL, Num
```

Trong vòng lặp `PrintLoop`, chương trình thực hiện các bước sau:

1. Gọi `printFibo` để in số Fibonacci hiện tại đang lưu trong `fibo_n1`.
2. In thêm một khoảng trắng thông qua chuỗi `Space_mgs`.
3. Gọi `Add_Array` để tính `fibo = fibo_n1 + fibo_n2`.
4. Gọi `copyArray` để sao chép `fibo_n2` sang `fibo_n1`.
5. Gọi `copyArray` lần nữa để sao chép `fibo` sang `fibo_n2`.

6. Dừng lệnh LOOP PrintLoop để giảm CX; nếu CX chưa bằng 0 thì tiếp tục vòng lặp.

Thủ tục Add_Array cộng hai mảng từ phần tử cuối về phần tử đầu. Thanh ghi DL được dùng để lưu phần nhớ. Nếu tổng của hai chữ số và phần nhớ lớn hơn hoặc bằng 10, chương trình trừ đi 10 và đặt DL = 1. Nếu tổng nhỏ hơn 10, DL được đặt lại bằng 0.

Thủ tục printFibo dùng thanh ghi DH để đánh dấu đã gặp chữ số khác 0 hay chưa. Các chữ số 0 ở đầu sẽ bị bỏ qua. Khi gặp chữ số khác 0 đầu tiên, chương trình bắt đầu in các chữ số còn lại. Nếu toàn bộ mảng đều bằng 0, chương trình in trực tiếp ký tự '0' để biểu diễn số Fibonacci đầu tiên.

3. Source code phiên bản sử dụng mảng

Source code đầy đủ của phiên bản sử dụng mảng được lưu trong file `ver1.asm`. Chương trình sử dụng ba mảng `fibo_n1`, `fibo_n2` và `fibo` để lưu trữ, cộng và cập nhật các số Fibonacci.

```
1  .MODEL SMALL
2  .STACK 100h
3
4  .DATA
5      Input_mgs    DB "Enter a 2-digits number: $"
6      Output_mgs   DB 13,10,"The first N Fibonacci numbers:",13,10,"$"
7      Error_mgs    DB 13,10,"Invalid N! Please enter 9 < N < 100.",13,10,"$"
8      Space_mgs    DB " $"
9
10     input1        DB 3,?,3 DUP(' ')
11     Num           DB ?
12
13     fibo_n1        DB 21 DUP(0)
14     fibo_n2        DB 21 DUP(0)
15     fibo           DB 21 DUP(0)
16
17 .CODE
18
19 Main PROC
20     MOV AX, @DATA
21     MOV DS, AX
22
23 InputAgain:
24     LEA BX, Input_mgs
25     LEA CX, input1
26     CALL Input
27     CALL Endline
28
29     LEA SI, input1 + 2
30     LEA BX, Num
31     CALL ConvertStrToNum
32
33     MOV AL, Num
34     CMP AL, 9
```

```

35     JLE InvalidInput
36
37     CMP AL, 100
38     JGE InvalidInput
39
40     JMP StartProgram
41
42 InvalidInput:
43     LEA DX, Error_mgs
44     MOV AH, 9
45     INT 21h
46
47     JMP InputAgain
48
49 StartProgram:
50     LEA DX, Output_mgs
51     MOV AH, 9
52     INT 21h
53
54     MOV fibo_n2 + 20, 1
55
56     XOR CX, CX
57     MOV CL, Num
58
59 PrintLoop:
60     LEA BX, fibo_n1
61     CALL printFibo
62
63     LEA DX, Space_mgs
64     MOV AH, 9
65     INT 21h
66
67     CALL Add_Array
68
69     LEA AX, fibo_n1
70     LEA DX, fibo_n2
71     CALL copyArray
72
73     LEA AX, fibo_n2
74     LEA DX, fibo
75     CALL copyArray
76
77     LOOP PrintLoop
78
79     CALL Endline
80
81     MOV AH, 4Ch
82     INT 21h
83 Main ENDP
84
85
86 printFibo PROC
87     PUSH AX

```

```

88     PUSH BX
89     PUSH CX
90     PUSH DX
91     PUSH SI
92
93     MOV SI, 0
94     MOV CX, 21
95     XOR DH, DH
96
97 loop_printFibo:
98     MOV DL, [BX + SI]
99     CMP DL, 0
100    JNE print_digit
101
102    CMP DH, 0
103    JE skip_zero
104
105 print_digit:
106    MOV DH, 1
107    ADD DL, '0'
108    MOV AH, 02h
109    INT 21h
110
111 skip_zero:
112    INC SI
113    LOOP loop_printFibo
114
115    CMP DH, 1
116    JE end_printFibo
117
118    MOV DL, '0'
119    MOV AH, 02h
120    INT 21h
121
122 end_printFibo:
123    POP SI
124    POP DX
125    POP CX
126    POP BX
127    POP AX
128    RET
129 printFibo ENDP
130
131
132 copyArray PROC
133    PUSH AX
134    PUSH BX
135    PUSH CX
136    PUSH SI
137    PUSH DI
138
139    MOV DI, AX
140    MOV SI, DX

```

```

141     MOV CX, 21
142
143 copy_loop:
144     MOV BL, [SI]
145     MOV [DI], BL
146
147     INC SI
148     INC DI
149
150     LOOP copy_loop
151
152     POP DI
153     POP SI
154     POP CX
155     POP BX
156     POP AX
157     RET
158 copyArray ENDP
159
160
161 Add_Array PROC
162     PUSH AX
163     PUSH BX
164     PUSH DX
165     PUSH SI
166
167     MOV SI, 20
168     MOV DL, 0
169
170 loop_add_array:
171     CMP SI, 0
172     JL end_loop_add_array
173
174     MOV AL, fibo_n1[SI]
175     MOV BL, fibo_n2[SI]
176
177     ADD AL, BL
178     ADD AL, DL
179
180     MOV DL, 0
181
182     CMP AL, 10
183     JL store_digit
184
185     SUB AL, 10
186     MOV DL, 1
187
188 store_digit:
189     MOV fibo[SI], AL
190
191     DEC SI
192     JMP loop_add_array
193

```



```

194 end_loop_add_array:
195     POP SI
196     POP DX
197     POP BX
198     POP AX
199     RET
200 Add_Array ENDP
201
202
203 ConvertStrToNum PROC
204     PUSH AX
205     PUSH BX
206     PUSH CX
207     PUSH SI
208
209     XOR AX, AX
210
211 Convert_loop:
212     MOV CL, [SI]
213     CMP CL, '$'
214     JE end_loop
215
216     MOV CH, 0
217
218     PUSH BX
219     MOV BX, 10
220     MUL BX
221
222     SUB CX, '0'
223     ADD AX, CX
224
225     POP BX
226     INC SI
227
228     JMP Convert_loop
229
230 end_loop:
231     MOV [BX], AL
232
233     POP SI
234     POP CX
235     POP BX
236     POP AX
237     RET
238 ConvertStrToNum ENDP
239
240
241 Input PROC
242     PUSH AX
243     PUSH BX
244     PUSH CX
245     PUSH DX
246     PUSH SI

```

```

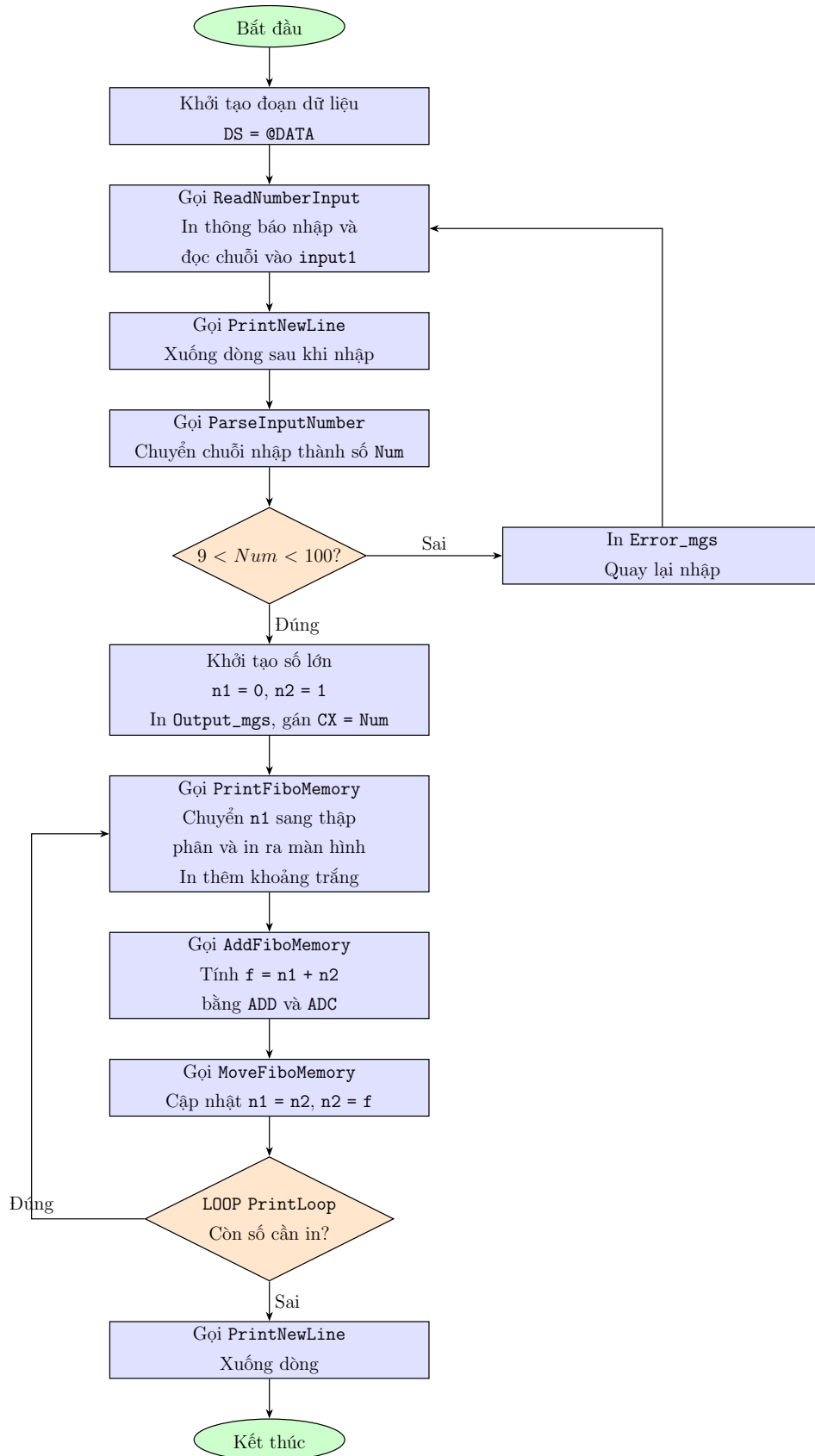
247
248     MOV DX, BX
249     MOV AH, 9
250     INT 21h
251
252     MOV DX, CX
253     MOV AH, 0Ah
254     INT 21h
255
256     MOV BX, CX
257     XOR SI, SI
258
259     MOV AL, [BX + 1]
260     MOV AH, 0
261     MOV SI, AX
262
263     MOV BYTE PTR [BX + SI + 2], '$'
264
265     POP SI
266     POP DX
267     POP CX
268     POP BX
269     POP AX
270     RET
271 Input ENDP
272
273
274 Endline PROC
275     PUSH AX
276     PUSH DX
277
278     MOV DL, 13
279     MOV AH, 2
280     INT 21h
281
282     MOV DL, 10
283     MOV AH, 2
284     INT 21h
285
286     POP DX
287     POP AX
288     RET
289 Endline ENDP
290
291 END Main

```

Listing 1. Source code phiên bản sử dụng mảng

III. Phiên bản 2: Chương trình sử dụng memory

1. Flowchart chương trình sử dụng memory



Hình 2. Flowchart chương trình Fibonacci sử dụng memory

2. Nguyên lý hoạt động

Ở phiên bản sử dụng memory, mỗi số Fibonacci được lưu bằng 5 biến **WORD** riêng lẻ. Mỗi **WORD** có kích thước 16-bit, vì vậy tổng dung lượng dùng để biểu diễn một số là:

$$5 \text{ WORD} = 5 \times 16 = 80 \text{ bit}$$

Dung lượng 80-bit đủ để lưu các số Fibonacci trong phạm vi yêu cầu của đề bài, vì $N < 100$.

Các biến **n1_0**, **n1_1**, **n1_2**, **n1_3**, **n1_4** dùng để lưu số Fibonacci hiện tại. Các biến **n2_0**, **n2_1**, **n2_2**, **n2_3**, **n2_4** dùng để lưu số Fibonacci kế tiếp. Các biến **f_0**, **f_1**, **f_2**, **f_3**, **f_4** dùng để lưu kết quả cộng $f = n1 + n2$. Ngoài ra, chương trình dùng các biến tạm **t_0**, **t_1**, **t_2**, **t_3**, **t_4** để hỗ trợ quá trình in số 80-bit ra dạng thập phân.

Sau khi khởi tạo đoạn dữ liệu bằng **DS = @DATA**, chương trình bắt đầu tại nhãn **InputAgain**. Tại đây, chương trình gọi thủ tục **ReadNumberInput** để in chuỗi **Input_mgs** và đọc chuỗi nhập từ bàn phím vào bộ đệm **input1**. Sau đó, chương trình gọi **PrintNewLine** để xuống dòng.

Chuỗi nhập được chuyển thành số nguyên bằng thủ tục **ParseInputNumber**. Thủ tục này duyệt từng ký tự trong bộ đệm, nhân kết quả hiện tại với 10 rồi cộng thêm chữ số mới. Giá trị cuối cùng được lưu vào biến **Num**.

Sau đó, chương trình kiểm tra điều kiện:

$$9 < \text{Num} < 100$$

Nếu **Num** không hợp lệ, chương trình nhảy đến **InvalidInput**, in thông báo lỗi **Error_mgs**, rồi quay lại **InputAgain**. Nếu hợp lệ, chương trình chuyển đến **StartProgram**.

Tại **StartProgram**, chương trình khởi tạo:

$$n1 = 0, \quad n2 = 1$$

Trong code, **n1** được biểu diễn bằng 5 biến **n1_0** đến **n1_4**; tất cả đều được gán 0. Số **n2** được biểu diễn bằng 5 biến **n2_0** đến **n2_4**; trong đó **n2_0 = 1**, các **WORD** còn lại bằng 0.

Tiếp theo, chương trình in thông báo **Output_mgs**, sau đó đưa số lượng phần tử Fibonacci cần in vào thanh ghi **CX**:

MOV CL, Num

Trong vòng lặp **PrintLoop**, chương trình thực hiện các bước:

1. Gọi **PrintFiboMemory** để in số Fibonacci hiện tại đang lưu trong **n1**.
2. In thêm một khoảng trắng bằng chuỗi **Space_mgs**.

3. Gọi `AddFiboMemory` để tính $f = n1 + n2$.
4. Gọi `MoveFiboMemory` để cập nhật $n1 = n2$ và $n2 = f$.
5. Dùng lệnh `LOOP PrintLoop` để giảm `CX`; nếu `CX` chưa bằng 0 thì tiếp tục vòng lặp.

Thủ tục `AddFiboMemory` thực hiện phép cộng số lớn theo từng `WORD`. Đầu tiên, chương trình cộng phần thấp nhất bằng lệnh `ADD`:

$$f_0 = n1_0 + n2_0$$

Các phần cao hơn được cộng bằng lệnh `ADC`, nhờ đó phần nhớ từ phép cộng trước được cộng tiếp sang `WORD` kế tiếp. Cách này giúp chương trình cộng được số lớn 80-bit trên vi xử lý 8086.

Thủ tục `MoveFiboMemory` thực hiện cập nhật dãy Fibonacci sau mỗi vòng lặp. Chương trình sao chép lần lượt các `WORD` của $n2$ sang $n1$, sau đó sao chép các `WORD` của f sang $n2$. Sau bước này, chương trình đã chuẩn bị xong hai số Fibonacci cho lần lặp kế tiếp.

Thủ tục `PrintFiboMemory` dùng để in số 80-bit ra màn hình dưới dạng thập phân. Trước tiên, chương trình sao chép giá trị của $n1$ sang các biến tạm `t_0` đến `t_4`. Nếu toàn bộ các biến tạm đều bằng 0, chương trình in trực tiếp ký tự '0'.

Nếu số cần in khác 0, chương trình liên tục gọi `DivideMemoryByTen`. Thủ tục này chia số 80-bit đang lưu trong các biến tạm cho 10, bắt đầu từ `WORD` cao nhất `t_4` xuống `WORD` thấp nhất `t_0`. Sau mỗi lần chia, phần dư chính là một chữ số thập phân. Vì các chữ số thu được theo thứ tự từ phải sang trái, chương trình đưa chúng vào stack bằng lệnh `PUSH`, sau đó lấy ra bằng `POP` để in theo đúng thứ tự từ trái sang phải.

3. Source code phiên bản sử dụng memory

Source code đầy đủ của phiên bản sử dụng memory được lưu trong file `ver2.asm`. Chương trình sử dụng các biến `WORD` riêng lẻ như `n1_0`, `n1_1`, ..., `n2_0`, `n2_1`, ... để biểu diễn số Fibonacci 80-bit.

```

1  .MODEL SMALL
2  .STACK 100h
3
4  .DATA
5      Input_mgs    DB "Enter a 2-digits number: $"
6      Output_mgs   DB 13,10,"The first N Fibonacci numbers:",13,10,"$"
7      Error_mgs    DB 13,10,"Invalid N! Please enter 9 < N < 100.",13,10,"$"
8      Space_mgs    DB " $"
9
10     input1        DB 3,?,3 DUP(' ')
11     Num            DB ?
12
13     n1_0           DW 0

```

```

14     n1_1      DW 0
15     n1_2      DW 0
16     n1_3      DW 0
17     n1_4      DW 0
18
19     n2_0      DW 1
20     n2_1      DW 0
21     n2_2      DW 0
22     n2_3      DW 0
23     n2_4      DW 0
24
25     f_0      DW 0
26     f_1      DW 0
27     f_2      DW 0
28     f_3      DW 0
29     f_4      DW 0
30
31     t_0      DW 0
32     t_1      DW 0
33     t_2      DW 0
34     t_3      DW 0
35     t_4      DW 0
36
37     Ten      DW 10
38
39     .CODE
40
41 Main PROC
42     MOV AX, @DATA
43     MOV DS, AX
44
45 InputAgain:
46     LEA BX, Input_mgs
47     LEA CX, input1
48     CALL ReadNumberInput
49     CALL PrintNewLine
50
51     LEA SI, input1 + 2
52     LEA BX, Num
53     CALL ParseInputNumber
54
55     MOV AL, Num
56     CMP AL, 9
57     JLE InvalidInput
58
59     CMP AL, 100
60     JGE InvalidInput
61
62     JMP StartProgram
63
64 InvalidInput:
65     LEA DX, Error_mgs
66     MOV AH, 9

```

```

67     INT 21h
68
69     JMP InputAgain
70
71 StartProgram:
72     MOV n1_0, 0
73     MOV n1_1, 0
74     MOV n1_2, 0
75     MOV n1_3, 0
76     MOV n1_4, 0
77
78     MOV n2_0, 1
79     MOV n2_1, 0
80     MOV n2_2, 0
81     MOV n2_3, 0
82     MOV n2_4, 0
83
84     LEA DX, Output_mgs
85     MOV AH, 9
86     INT 21h
87
88     XOR CX, CX
89     MOV CL, Num
90
91 PrintLoop:
92     CALL PrintFiboMemory
93
94     LEA DX, Space_mgs
95     MOV AH, 9
96     INT 21h
97
98     CALL AddFiboMemory
99     CALL MoveFiboMemory
100
101     LOOP PrintLoop
102
103     CALL PrintNewLine
104
105     MOV AH, 4Ch
106     INT 21h
107 Main ENDP
108
109
110 AddFiboMemory PROC
111     PUSH AX
112
113     MOV AX, n1_0
114     ADD AX, n2_0
115     MOV f_0, AX
116
117     MOV AX, n1_1
118     ADC AX, n2_1
119     MOV f_1, AX

```

```

120
121     MOV AX, n1_2
122     ADC AX, n2_2
123     MOV f_2, AX
124
125     MOV AX, n1_3
126     ADC AX, n2_3
127     MOV f_3, AX
128
129     MOV AX, n1_4
130     ADC AX, n2_4
131     MOV f_4, AX
132
133     POP AX
134     RET
135 AddFiboMemory ENDP
136
137
138 MoveFiboMemory PROC
139     PUSH AX
140
141     MOV AX, n2_0
142     MOV n1_0, AX
143
144     MOV AX, n2_1
145     MOV n1_1, AX
146
147     MOV AX, n2_2
148     MOV n1_2, AX
149
150     MOV AX, n2_3
151     MOV n1_3, AX
152
153     MOV AX, n2_4
154     MOV n1_4, AX
155
156     MOV AX, f_0
157     MOV n2_0, AX
158
159     MOV AX, f_1
160     MOV n2_1, AX
161
162     MOV AX, f_2
163     MOV n2_2, AX
164
165     MOV AX, f_3
166     MOV n2_3, AX
167
168     MOV AX, f_4
169     MOV n2_4, AX
170
171     POP AX
172     RET

```



```

173 MoveFiboMemory ENDP
174
175
176 PrintFiboMemory PROC
177     PUSH AX
178     PUSH BX
179     PUSH CX
180     PUSH DX
181
182     MOV AX, n1_0
183     MOV t_0, AX
184
185     MOV AX, n1_1
186     MOV t_1, AX
187
188     MOV AX, n1_2
189     MOV t_2, AX
190
191     MOV AX, n1_3
192     MOV t_3, AX
193
194     MOV AX, n1_4
195     MOV t_4, AX
196
197     CMP t_0, 0
198     JNE ConvertPrint
199
200     CMP t_1, 0
201     JNE ConvertPrint
202
203     CMP t_2, 0
204     JNE ConvertPrint
205
206     CMP t_3, 0
207     JNE ConvertPrint
208
209     CMP t_4, 0
210     JNE ConvertPrint
211
212     MOV DL, '0'
213     MOV AH, 2
214     INT 21h
215
216     JMP FinishPrintMemory
217
218 ConvertPrint:
219     XOR CX, CX
220
221 DivideLoop:
222     CALL DivideMemoryByTen
223
224     ADD DL, '0'
225     MOV DH, 0

```

```

226     PUSH DX
227
228     INC CX
229
230     CMP t_0, 0
231     JNE DivideLoop
232
233     CMP t_1, 0
234     JNE DivideLoop
235
236     CMP t_2, 0
237     JNE DivideLoop
238
239     CMP t_3, 0
240     JNE DivideLoop
241
242     CMP t_4, 0
243     JNE DivideLoop
244
245 PrintDigitLoop:
246     POP DX
247     MOV AH, 2
248     INT 21h
249
250     LOOP PrintDigitLoop
251
252 FinishPrintMemory:
253     POP DX
254     POP CX
255     POP BX
256     POP AX
257     RET
258 PrintFiboMemory ENDP
259
260
261 DivideMemoryByTen PROC
262     PUSH AX
263     PUSH BX
264
265     MOV BX, Ten
266     XOR DX, DX
267
268     MOV AX, t_4
269     DIV BX
270     MOV t_4, AX
271
272     MOV AX, t_3
273     DIV BX
274     MOV t_3, AX
275
276     MOV AX, t_2
277     DIV BX
278     MOV t_2, AX

```

```

279
280     MOV AX, t_1
281     DIV BX
282     MOV t_1, AX
283
284     MOV AX, t_0
285     DIV BX
286     MOV t_0, AX
287
288     POP BX
289     POP AX
290     RET
291 DivideMemoryByTen ENDP
292
293
294 ParseInputNumber PROC
295     PUSH AX
296     PUSH BX
297     PUSH CX
298     PUSH SI
299
300     XOR AX, AX
301
302 ParseLoop:
303     MOV CL, [SI]
304     CMP CL, '$'
305     JE FinishParse
306
307     MOV CH, 0
308
309     PUSH BX
310     MOV BX, 10
311     MUL BX
312
313     SUB CX, '0'
314     ADD AX, CX
315
316     POP BX
317     INC SI
318
319     JMP ParseLoop
320
321 FinishParse:
322     MOV [BX], AL
323
324     POP SI
325     POP CX
326     POP BX
327     POP AX
328     RET
329 ParseInputNumber ENDP
330
331

```

```

332 ReadNumberInput PROC
333     PUSH AX
334     PUSH BX
335     PUSH CX
336     PUSH DX
337     PUSH SI
338
339     MOV DX, BX
340     MOV AH, 9
341     INT 21h
342
343     MOV DX, CX
344     MOV AH, 0Ah
345     INT 21h
346
347     MOV BX, CX
348     XOR SI, SI
349
350     MOV AL, [BX + 1]
351     MOV AH, 0
352     MOV SI, AX
353
354     MOV BYTE PTR [BX + SI + 2], '$'
355
356     POP SI
357     POP DX
358     POP CX
359     POP BX
360     POP AX
361     RET
362 ReadNumberInput ENDP
363
364
365 PrintNewLine PROC
366     PUSH AX
367     PUSH DX
368
369     MOV DL, 13
370     MOV AH, 2
371     INT 21h
372
373     MOV DL, 10
374     MOV AH, 2
375     INT 21h
376
377     POP DX
378     POP AX
379     RET
380 PrintNewLine ENDP
381
382 END Main

```

Listing 2. Source code phiên bản sử dụng memory

IV. So sánh hai phiên bản chương trình

Tiêu chí	Phiên bản dùng mảng	Phiên bản dùng memory
Cách lưu trữ	Mỗi số Fibonacci được lưu bằng một mảng gồm 21 phần tử.	Mỗi số Fibonacci được lưu bằng 5 biến WORD riêng lẻ.
Đơn vị lưu trữ	Mỗi phần tử lưu một chữ số thập phân.	Mỗi WORD lưu 16 bit dữ liệu nhị phân.
Cách tính toán	Cộng từng chữ số từ phải sang trái.	Cộng từng WORD từ thấp đến cao.
Xử lý nhớ	Tự xử lý nhớ bằng biến DL.	Dùng Carry Flag thông qua lệnh ADD và ADC.
Cách in kết quả	In trực tiếp từng chữ số trong mảng.	Phải chia số 80-bit nhiều lần cho 10 để chuyển sang thập phân.
Tốc độ tính toán	Chậm hơn vì xử lý từng chữ số.	Nhanh hơn vì xử lý theo từng khối 16 bit.
Khả năng mở rộng	Dễ mở rộng bằng cách tăng kích thước mảng.	Muốn mở rộng phải khai báo thêm các biến WORD.

V. Kết quả mô phỏng

Link Google Drive:

<https://drive.google.com/drive/folders/1ugN2Fx7-xljKrflVMXAIHurBzUW5XtOW?usp=sharing>